

What is RAG?

LESSON 10 · MODULE 3 — KNOWLEDGE & MEMORY

What you'll learn

- The failure mode RAG is designed to fix: confident hallucinations on private/internal data
- The three-step RAG flow in Lyzr: create a knowledge base, add a doc, pass it to `agent.run()`
- Why the knowledge base is attached at runtime, not at agent creation
- When to use RAG and — equally important — when to skip it

Key concepts

The failure mode without RAG. Ask a model about your private B2B SaaS company and it will pattern-match from generic setup docs and ship a confident guess. Even if you instruct the agent to "only answer from documentation provided," without that documentation it will either refuse (good) or hallucinate (bad). The instruction *forces* the gap to be visible.

TAKEAWAY

The failure mode without RAG.

Three-step RAG in Lyzr.

TAKEAWAY

Three-step RAG in Lyzr.

1. Create a knowledge base — defaults handle the rest (vector store: Qdrant; embedding: OpenAI `text-embedding-3-large` ; persisted to your Lyzr account).
2. Add a document — `add_text` , `add_pdf` , `add_docx` , or `add_website` .
3. Pass it to `agent.run(knowledge_bases=kb)` at call time.

The knowledge base is attached at runtime. Same agent can answer with different knowledge bases on different calls. Useful when you've got a routing layer in front (think back to lesson 06).

TAKEAWAY

The knowledge base is attached at runtime.

The model still adds value on top of retrieval. Retrieval gives the agent grounded facts. The LLM shapes them — numbered walkthroughs, separated sections, structured walkthrough. Don't think of RAG as "the LLM goes away" — think of it as "the LLM stops guessing about facts it doesn't know."

TAKEAWAY

The model still adds value on top of retrieval.

When to use RAG. Your data is private or internal. The data changes over time. The data is recent enough to be past the model's training cutoff. You need source attribution. The corpus is too large to dump into the prompt.

TAKEAWAY

When to use RAG.

When to skip RAG. Data fits in the prompt window (modern flagship models take hundreds of thousands to millions of tokens — if your knowledge base is one PDF, just paste it in). Data lives in a structured database — SQL or an API call beats vector search on structured data,

TAKEAWAY

When to skip RAG.

so use a tool, not a knowledge base. The model already knows the territory well — adding RAG would slow you down and add nothing.

The honest version. RAG is the right tool when your data is external, dynamic, or large. Otherwise reach for prompt context, a tool call, or a system prompt. Don't add infrastructure you don't need.

TAKEAWAY

The honest version.

Code shown in this lesson

PYTHON

```
agent = studio.create_agent(
    name="veristack-support",
    provider="gpt-4o",
    role="VeriStack customer support agent",
    goal="answer admin questions accurately from documentation only",
    instructions=(
        "Only answer from documentation provided in context. "
        "If the answer isn't in the docs, reply: "
        "'I don't have that information in the documentation I can see.'"
    ),
)

# Without RAG – the agent refuses cleanly (or would hallucinate without strict instructions)
agent.run("Our VeriStack workspace needs SSO with Okta. What are the steps?")
```

PYTHON

```
# Step 1 – create a knowledge base
kb = studio.create_knowledge_base(name="veristack-admin-docs")

# Step 2 – add a document
kb.add_text("""
    VeriStack SSO with Okta – admin steps...
    [...documentation snippet here...]
""")

# Step 3 – pass to agent.run
response = agent.run(
    "Our VeriStack workspace needs SSO with Okta. What are the steps?",
    knowledge_bases=[kb],
)
print(response.response)
```

TRY THIS

- 1 Build a knowledge base with one fake fact — *"VeriStack supports SSO with Okta via SAML 2.0."* Query the same agent twice — once *without* the knowledge base attached, once *with*. Verify it refuses cleanly without it and quotes the fact verbatim with it.
- 2 Attach the *same* knowledge base to two agents — one with `role="customer support agent"`, one with `role="senior platform engineer"`. Same question, same knowledge base, two different `role`s. Note how the role shapes the *form* of the answer even when the facts are identical.